

PFM: Serving Current Personal Facts Under On-Device Latency Budgets

Anonymous submission

Abstract

Persistent memory is useful to an on-device language assistant only when the retrieved fact concerns the intended person, remains current, and reaches the prompt before generation begins. We formalize *deadline-constrained personal fact retrieval*, which couples identity, temporal validity, prompt capacity, and serving latency. Personal Fact Memory (PFM) represents prompt-ready facts with entity, participant, and temporal metadata, applies keyed supersession to recognized updates, and serves them with field-weighted lexical retrieval and bounded local association while keeping model inference off the request path. In replay through a deployed macOS autocomplete assistant, PFM retrieves the target fact in 24 of 25 scenarios and resolves all 8 identity probes, versus 5 of 8 without participant metadata. On a 240-query update benchmark at $B=800$, keyed supersession has zero observed stale exposure, whereas a keyless store exposes stale facts on 66.2% of queries. PFM achieves .875 clean retrieval with 0.06 ms median fresh-serving latency; validity-filtered BM25 reaches .758, while all validity-agnostic lexical, dense, and hybrid baselines remain below .27. In frozen-7B replay, PFM produces 92.5% correct completions with no observed stale or wrong-person outputs, while plain BM25 yields 51.7% correct and 35.8% stale completions.

Introduction

An on-device autocomplete assistant can write fluent text while knowing nothing about the user’s current facts. Suppose a user replying to Alex types “The concert is at.” The useful continuation is the time Alex and the user most recently agreed on, not a plausible time from the model’s parametric knowledge. Supplying that fact requires the memory layer to identify the intended person, account for subsequent revisions, and return the result before generation starts. Figure 1 shows this setting in the deployed autocomplete assistant studied here.

Persistent-memory systems increasingly make long interaction histories retrievable through extracted records, temporal graphs, memory tiers, and associative search (Chhikara et al. 2025; Rasmussen et al. 2025; Gutiérrez et al. 2024; Packer et al. 2023). Benchmarks such as LongMemEval ask whether information from earlier conversations can be recovered (Wu et al. 2024). Continuous personal assistance imposes a stricter serving requirement. A semantically relevant fact can still belong to the wrong person, have been

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

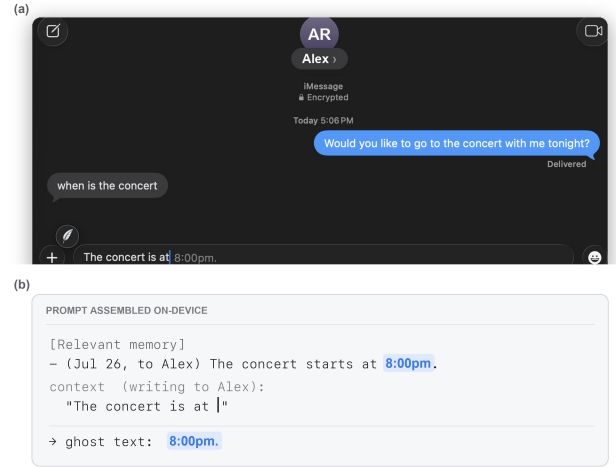


Figure 1: PFM in the deployed autocomplete assistant, with the recipient pseudonymized. The user types “The concert is at.” PFM supplies the remembered time in a compact [Relevant memory] block before the frozen language model generates the completion.

superseded by a later update, or arrive after the host model has begun generation. On a personal device, these errors arise under the same compute and latency budget as the language model itself.

We call this problem *deadline-constrained personal fact retrieval*. Each request supplies local context, interaction participants, an evolving personal fact store, a prompt budget, and a serving deadline. A retrieval is useful only if it supplies a relevant fact for the intended identity while that fact remains valid and before the deadline. We distinguish three corresponding failures: *misattribution*, when the fact belongs to the wrong person or entity; *staleness*, when a superseded value is surfaced as current; and *lateness*, when useful context misses the serving deadline.

We develop **Personal Fact Memory (PFM)** for this regime. PFM stores prompt-ready atomic facts with explicit entity and participant fields and temporal version information. Field-weighted lexical retrieval uses identities from the current interaction, while a bounded local entity graph sup-

plies an additional association signal when the typed text omits the target entity. Model-dependent extraction and index maintenance run asynchronously. Completion-time serving uses only in-memory state and immutable snapshots, so no embedding model or language model is invoked on the retrieval path. Recognized revisions share a mutable slot key, allowing PFM to remove the superseded value from the active index while retaining its history.

The experiments separate identity, validity, and serving time rather than collapsing them into retrieval accuracy. In deployed-path replay, PFM places the target fact in 24 of 25 prompts and resolves 8 of 8 identity probes, compared with 5 of 8 without participant metadata. A controlled 240-query update benchmark isolates temporal versioning: at $B=800$, keyed supersession achieves 1.000 current-value recall with zero observed stale exposure, while the keyless configuration exposes a stale value on 66.2% of queries. On the same corpus and device, validity-filtered BM25 reaches .758 clean retrieval and PFM .875, whereas every validity-agnostic baseline remains below .27; MiniLM attains .975 current-value recall but .779 stale exposure and 15.4 ms median query latency. In frozen-7B replay, PFM produces 92.5% correct completions with no observed stale or wrong-person outputs, while plain BM25 produces 51.7% correct and 35.8% stale completions. These experiments measure retrieval and generated text, not live user benefit.

The paper makes three contributions.

1. **Problem formulation.** We formalize deadline-constrained personal fact retrieval and separate misattribution, staleness, and lateness under prompt and serving-time budgets.
2. **Memory architecture.** PFM combines participant-aware fact representation, keyed temporal supersession, asynchronous construction, and a model-free serving path. We give conditions under which keyed updates maintain a unique active value and the serving protocol bounds memory-induced waiting.
3. **System evidence.** We evaluate PFM in a deployed auto-complete path, a controlled update benchmark, matched lexical, dense, and hybrid baselines, and frozen-model replay. The results isolate the roles of participant attribution, temporal validity, and serving latency in both prompt-side and completion-level errors.

Related Work

Personal and evolving memory. Long-term agent memory has been implemented through memory tiers, extracted facts, linked notes, and graph retrieval. MemGPT pages between memory tiers (Packer et al. 2023), Mem0 extracts and updates persistent facts (Chhikara et al. 2025), A-MEM stores linked atomic notes (Xu et al. 2026), HippoRAG retrieves over an entity graph (Gutiérrez et al. 2024), and EMG-RAG constructs an editable memory graph from smartphone data (Wang et al. 2024). Temporal evolution is also explicit in recent systems: Zep maintains a temporal knowledge graph (Rasmussen et al. 2025), APEX-MEM resolves conflicts in an append-only property graph (Banerjee et al. 2026), Temporal Semantic Memory models event time and durative states

(Su et al. 2026), and Memora evaluates reliance on obsolete information after updates (Xia et al. 2026). PFM addresses a different boundary condition: identity attribution, current validity, and availability before generation must hold on the same request.

Efficient memory serving. Memory cost is increasingly part of system design. Lightweight LLM Agent Memory separates online retrieval from offline consolidation under a fixed retrieval budget (Zhang et al. 2026); AME co-designs vector-memory querying, insertion, and maintenance for mobile hardware (Zhao et al. 2025); AgentIR decides when a dense retrieval channel is worth its cost (Yuan, Zhang, and Nazarian 2026); and system studies characterize construction, retrieval, and freshness costs across agent execution (Omri et al. 2026; Zhou et al. 2026). PFM fixes completion-time latency as part of the retrieval problem: serving uses participant metadata and active temporal state without model inference, and a request can fall back to a published snapshot when fresh retrieval exceeds its waiting budget.

Evaluation of long-term memory. LoCoMo and Long-MemEval test recovery and reasoning over extended histories (Maharana et al. 2024; Wu et al. 2024); recent work adds temporal validity, obsolete-memory penalties, retrieval efficiency, and systems cost (Xia et al. 2026; Su et al. 2026; Huang et al. 2025; Omri et al. 2026). Autocomplete adds interaction structure and a hard serving boundary: participant context can disambiguate similar facts, prompt space is limited, and memory must arrive before the completion request proceeds. Our evaluation therefore reports identity errors, stale exposure, target availability, and serving latency separately.

Problem Formulation

We consider a single user whose personal memory evolves as documents, messages, and other observations arrive over time. At time t , let $\mathcal{M}_t$ denote the facts retained by the memory layer, including active facts and facts preserved only as history. A completion request consists of typing context c_t , a participant set P_t , a serving deadline Δ_t , a maximum number of injected facts k , and a character budget B . The deadline is the latest time at which retrieved context can be incorporated before the host model begins generation.

Definition 1 (Personal fact). *A stored fact is a tuple*

$$f = (z_f, E_f, P_f, \kappa_f, a_f, b_f, \ell_f), \quad (1)$$

where z_f is the prompt-ready text, E_f is a set of normalized entities, P_f is the participant set associated with the source, κ_f is an optional key for a mutable semantic slot, $[a_f, b_f]$ is the interval during which the system treats the fact as active, and ℓ_f is its prompt length. We set $b_f = \infty$ when a fact has not been superseded. The active view at time t is $\mathcal{A}_t = \{f \in \mathcal{M}_t : a_f \leq t < b_f\}$.

The active interval records the state of the memory system; it does not assert that the stored proposition is true in the world. An extractor may fail to recognize that a new observation revises an earlier one. Semantic validity must therefore be evaluated separately from the system’s active view.

A retrieval policy π maps $(\mathcal{A}_t, c_t, P_t)$ to an ordered set $S_{\pi,t} \subseteq \mathcal{A}_t$ and incurs serving latency $L_{\pi,t}$. The returned set must satisfy

$$|S_{\pi,t}| \leq k, \quad \sum_{f \in S_{\pi,t}} \ell_f \leq B, \quad (2)$$

and must be available by Δ_t to affect the current completion. The prompt constraints limit the amount of memory supplied to the language model, while the deadline limits the time available to obtain it.

For evaluation, let $R_t(f) = 1$ when fact f is relevant to request t , $I_t(f) = 1$ when its entity and participant attribution matches the intended identity of the request, and $V_t(f) = 1$ when the proposition remains semantically valid at time t . These labels describe ground truth and need not be observable by the retrieval policy. Define

$$\mathcal{G}_t = \{f \in \mathcal{M}_t : R_t(f) = 1, I_t(f) = 1, V_t(f) = 1\} \quad (3)$$

as the set of facts that are relevant, correctly attributed, and valid for request t .

Definition 2 (Deadline-constrained personal fact retrieval). *Retrieval succeeds on request t when at least one fact in $\mathcal{G}_t$ is returned within the serving deadline while satisfying the prompt constraints. Define correct-by-deadline availability as*

$$Y_{\pi,t}^{\text{db}} = \mathbf{1}\{S_{\pi,t} \cap \mathcal{G}_t \neq \emptyset\} \mathbf{1}\{L_{\pi,t} \leq \Delta_t\}. \quad (4)$$

Misattribution occurs when a retrieved fact is relevant in content but belongs to the wrong identity, $E_{\pi,t}^{\text{id}} = \mathbf{1}\{\exists f \in S_{\pi,t} : R_t(f) = 1, I_t(f) = 0\}$. Staleness occurs when a retrieved fact concerns the intended identity but is no longer valid, $E_{\pi,t}^{\text{stale}} = \mathbf{1}\{\exists f \in S_{\pi,t} : R_t(f) = 1, I_t(f) = 1, V_t(f) = 0\}$. Lateness occurs when retrieval misses the serving deadline, $E_{\pi,t}^{\text{late}} = \mathbf{1}\{L_{\pi,t} > \Delta_t\}$. The three indicators need not be mutually exclusive. A prompt may contain both a current value and a superseded value, so successful fact retrieval does not by itself imply temporal correctness.

For evaluations with complete identity and validity labels, we also define clean correct-by-deadline retrieval,

$$Y_{\pi,t}^{\text{clean}} = Y_{\pi,t}^{\text{db}} \mathbf{1}\{E_{\pi,t}^{\text{id}} = 0\} \mathbf{1}\{E_{\pi,t}^{\text{stale}} = 0\}. \quad (5)$$

This stricter outcome requires the target fact to be available by the deadline without exposing a relevant wrong-identity or stale fact on the same request.

Keyed temporal updates. Consider a mutable semantic slot with key κ . Suppose every recognized revision to that slot receives the same key and updates are processed serially. When a new fact f' with key κ is inserted at time s , the update rule sets $a_{f'} = s$ and $b_{f'} = \infty$. If an active predecessor f with the same key exists, the rule also sets $b_f = s$. The predecessor remains in $\mathcal{M}_t$ as history but is removed from the active view.

Proposition 1 (Unique active value under keyed supersession). *Fix a key κ . If updates with key κ are processed serially by the rule above and the store initially contains at most one active fact with that key, then for every time t ,*

$$|\{f \in \mathcal{A}_t : \kappa_f = \kappa\}| \leq 1. \quad (6)$$

The claim follows by induction over keyed updates because a new value becomes active at the same instant that the unique active predecessor, if any, is closed. The full proof is given in the supplementary material. The result depends on correct key assignment; it does not guarantee that the extractor identifies every revision or that the newest observation is semantically correct.

Personal Fact Memory

PFM separates memory construction from memory serving. Construction begins after the system observes a document and may include fact extraction, deduplication, temporal updates, graph maintenance, and pruning. These operations run asynchronously. Serving operates on the resulting in-memory state and does not call an embedding model or language model.

Memory Construction

Each extracted fact carries prompt text, normalized entities, source participants, and an optional key for a mutable semantic slot, as defined in the previous section. In the deployed assistant, the participant field is obtained from the interaction context in which the source document is observed. For supported messaging and mail applications, the active window identifies the interlocutor independently of the words in the document. This metadata is stored with the fact and later used during retrieval.

When a newly extracted fact receives the same slot key as an active fact, PFM applies keyed supersession and removes the predecessor from the active view. Supersession is checked before near-duplicate admission control: a revision that reuses the sentence surface with only the value changed must update the slot, not be absorbed as a re-observation of the old fact. Only an exactly repeated sentence reinforces the current value. Facts for which no reliable slot key is available remain separate observations. Proposition 1 therefore applies to recognized revisions, not to arbitrary pairs of contradictory text.

PFM maintains a fielded inverted index over active facts, with separate fields for prompt text, entities, and participants. It also maintains an entity co-occurrence graph whose edges summarize associations observed in the user’s fact store. Index and graph updates occur on the construction path. Historical facts may be retained for provenance, but inactive facts do not enter retrieval.

The implementation also records a fact class and a usage score. Facts are labeled episodic, semantic, or profile for retention purposes. Profile facts are pinned, while other facts may be pruned as recency and use decline. The usage score increases when a completion that used the fact is accepted and decays over time. These quantities are used as ranking and retention signals; the paper does not treat them as evidence for a cognitive model of human memory.

Retrieval and Ranking

For a completion request, PFM constructs a focal query from the tokens preceding the cursor and from the current participant set. Token weights decay with distance from the cursor

as λ^d , with $\lambda = 0.95$ in the deployed configuration. Participant terms receive an additional weight of 1.5, and the query retains at most 12 terms ranked by weighted inverse document frequency. Limiting the query size bounds the amount of query-side work before index traversal.

The lexical retriever uses BM25F (Robertson, Zaragoza, and Taylor 2004; Robertson and Zaragoza 2009) over the text, entity, and participant fields. The deployed field weights are 1.0, 2.5, and 2.0, respectively. Participant weighting distinguishes facts whose text is similar but whose source interactions involve different people. For example, two messages that both say “deadline moved to Friday” can be separated when one was sent to Alice and the other to Bob.

Lexical retrieval is supplemented by bounded one-hop propagation over the entity graph. Let Q_t be the seed entities recovered from the query and participant metadata, and let $w_{eg}(t)$ be the current weight between entities e and g . For fact f with entity set E_f , PFM computes

$$s_{\text{assoc}}(f \mid Q_t) = \sum_{e \in Q_t} \sum_{g \in E_f} w_{eg}(t), \quad (7)$$

with fixed limits on the number of seeds and neighbors traversed. The graph score provides an association signal when the current text names an entity connected to the target fact but does not name the target directly. This differs from full graph ranking methods such as HippoRAG (Gutiérrez et al. 2024), which perform broader propagation for open-domain retrieval.

The deployed ranker combines lexical relevance, recency, prior use, and graph association. Let Δt_f denote the age of fact f , let $h_f(t)$ denote its decaying usage score, and let T_{rec} be the recency half-life. The score is

$$\text{score}(f) = \text{BM25F}(q_t, f) \left(\varepsilon + (1 - \varepsilon) 2^{-\Delta t_f / T_{\text{rec}}} \right) \cdot (1 + \eta h_f(t)) + \gamma s_{\text{assoc}}(f \mid Q_t), \quad (8)$$

where ε , η , and γ are fixed parameters. The recency floor ε allows an old fact to remain retrievable when its lexical evidence is strong. The usage term gives more weight to facts that have previously contributed to accepted completions. The experiments ablate these components separately rather than assuming that each improves retrieval.

PFM ranks candidate facts by (8) and inserts them in score order until either k facts or B characters have been selected; this greedy rule is the deployed policy. A budget-aware alternative that reallocates prompt space across facts of unequal length, together with its analysis, is presented in the supplementary material.

Serving Protocol

PFM supports snapshot serving and fresh retrieval with bounded waiting. In snapshot mode, a background thread retrieves candidate facts during typing pauses and publishes an immutable result by atomic pointer replacement. When a completion request arrives, the serving thread reads the latest published snapshot and proceeds without waiting for a new retrieval. In fresh mode, the request starts a new retrieval and waits for at most δ_t before falling back to the latest snapshot.

The waiting budget δ_t is chosen below the request deadline Δ_t .

Let r be an upper bound on the time required to read a published snapshot after the serving thread is scheduled. Assume that published snapshots are immutable, publication uses an atomic pointer replacement, and the serving path acquires no lock held by memory construction or background retrieval.

Proposition 2 (Bound on memory-induced waiting). *Under the assumptions above, snapshot mode adds at most r time to the completion path. Fresh mode adds at most $\delta_t + r$. These bounds do not depend on the duration of fact extraction, index maintenance, graph updates, or pruning.*

The bound follows because snapshot mode performs only the bounded snapshot read, while fresh mode waits for at most δ_t before using a fresh result or the latest snapshot. The full proof is given in the supplementary material. Proposition 2 bounds delay introduced by memory serving; it does not guarantee that a fresh result is available by the deadline or that the retrieved facts are correct.

Pruning. Periodic pruning removes nonprofile facts whose recency and usage scores fall below a configured threshold and deletes the corresponding graph state. This policy reduces inactive or rarely used state in practice, but a threshold rule alone does not impose a deterministic upper bound on memory size under arbitrary arrival patterns. Storage footprint is therefore measured empirically rather than claimed as a consequence of the pruning rule.

Deployment Setting

PFM is integrated into a commercially deployed macOS autocomplete assistant that offers inline completions in text fields across applications: a greyed continuation appears at the cursor, the user accepts the full completion or a single word, and continued typing dismisses it. This makes the serving constraint concrete because memory must be incorporated before the prompt is handed to the host model, while acceptance provides an observable usage signal. For supported messaging and mail applications the frontmost window title identifies the interlocutor, which PFM records as participant metadata when a sent message is observed and reads again when a later completion is issued; this instantiates the participant set P_t from the interaction context rather than from the words being typed, letting two lexically similar facts be separated when they arose in conversations with different people.

The assistant captures sent text after a send-confirmation event and on application or window transitions, and passes observations asynchronously to memory construction, so extraction, indexing, temporal updates, and pruning never delay the triggering request. Capture excludes password and secure-entry fields, suppresses credential-management applications, and drops secret-like content (API keys, access tokens); the fact store stays on the device. At completion time, PFM serializes the selected facts into a compact [Relevant memory] block in the user-context portion of the prompt, capped at B characters so memory cannot

crowd out live context; in snapshot mode the published block is stable across a typing burst, which avoids changing the prompt prefix at every keystroke and can preserve host prompt caching. Acceptance is recorded only when the user adopts a completion and drives the retention and ranking mechanisms above; we treat it as a usage signal, not as evidence that a particular retrieved fact caused the acceptance.

Evaluation

We evaluate PFM at the memory layer using replay through the deployed assistant’s capture and serving paths, and then extend the evaluation along three axes that replay alone cannot support: temporal correctness on a controlled update benchmark, comparison against standard retrieval families under identical corpus, device, and deadline budgets, and completion-level effects through a frozen local language model. Systems-scaling measurements and a budget-aware selection study appear in the supplementary material. The primary replay metric is target fact availability, defined as the fraction of requests for which the prompt contains the ground-truth tokens. This metric is weaker than the clean correct-by-deadline outcome in Eq. (5) because the replay corpus does not provide exhaustive identity and validity labels for every retrieved candidate. We therefore report targeted identity and stale co-injection probes separately rather than constructing an unsupported clean aggregate.

Replay Setup

A scripted 28-day sequence of sent messages is ingested through the deployed capture path, including the same participant metadata used by the application. The small harness contains 9 captured messages producing 8 indexed facts and 7 completion scenarios. The large harness begins with 31 hand-written seed facts, adds 16 chit-chat messages and 240 synthetic filler messages (287 captured messages in total), and produces 270 indexed facts. The 25 large-harness scenarios span direct recall (20), associative recall where the prefix omits the target entity (2), a conflicting update (1), a CJK entity (1), and one deliberate extraction failure (a sentence lacking a signal verb). Retrieval uses $k = 6$ and an 800-character prompt budget.

Target Fact Availability

With memory enabled, the target tokens reach the prompt in 6 of 7 small-harness scenarios and 24 of 25 (96%) large-harness scenarios; with memory disabled, none of the target facts are present because the live typing context does not contain them (0/7 and 0/25). On the large harness, availability is 20/20 for direct recall, 2/2 associative, 1/1 conflicting-update, and 1/1 CJK; the designed extraction failure accounts for the remaining miss. Availability records only whether the ground-truth tokens appear in the constructed prompt; it does not by itself rule out stale or misattributed co-injection.

The write path admits 30 of 31 seed facts and none of the 16 chit-chat messages in this scripted corpus. The missed seed is a sentence without the signal verb required by the deployed rule extractor. These counts characterize extraction on the replay corpus only and should not be interpreted as general extraction precision or recall.

Arm	B	Current	Stale	Co-inj.	Clean
Keyed	800	1.000	.000	.000	.875
Keyless	800	.812	.662	.525	.283
Keyed	240	.896	.000	.000	.896
Keyless	240	.667	.388	.250	.417

Table 1: Update benchmark (240 labeled queries, filler corpus; the no-filler corpus is within one point on every entry). Current = current-value recall; Stale = stale-fact exposure; Co-inj. = current and superseded values in the same prompt; Clean = current value present with no stale or wrong-person exposure. The 95 percent upper bound on keyed stale exposure is 1.6 percent.

Identity Attribution

The overall availability metric does not test whether lexically similar memories are assigned to the correct person. We therefore construct four pairs of near-identical facts associated with different participants and query each pair from both participant contexts, yielding eight identity probes. PFM resolves all 8 probes when participant metadata is retained. Removing the participant field and its retrieval weight reduces performance to 5/8. Overall availability on the original 25 scenarios remains 24/25 in both conditions, showing why identity-specific probes are needed in addition to aggregate availability.

Temporal Correctness under Keyed Revision

The replay harness has a single conflicting-update scenario, too small to estimate temporal error rates. We therefore construct a controlled update benchmark of 160 revision chains over (entity, slot) pairs (36 people, 20 projects, six mutable slot types), varying revision depth (0–4), paraphrase, verbatim revision, cancellation versus replacement, and shared-first-name identity ambiguity. Every value token is globally unique, so exposure reduces to exact normalized containment. This yields 240 labeled queries per corpus, evaluated at two corpus sizes (with and without 1,000 filler messages) and two prompt budgets, with 95 percent Wilson intervals; the full construction protocol is in the supplement.

Two arms isolate the supersession mechanism. The *keyed* arm uses a key-emitting rule extractor (entity and slot noun plus a type-consistent value) with supersession applied before near-duplicate admission, as described in the construction section. The *keyless* arm ingests the same sentences with keys stripped, which is the configuration whose limitation the conflicting-update scenario exposed. Extraction coverage is identical by construction, so differences are attributable to temporal versioning alone.

Table 1 shows that keyed supersession changes temporal behavior end to end. The keyed arm recalls the current value on all 240 queries and never exposes a superseded value, so co-injection is eliminated rather than reduced. The keyless arm exposes a stale value on 66.2 percent of queries and co-injects on 52.5 percent, and recency ordering does not repair this because both values often survive the top- k cut. Three further effects are detailed in the supplement:

Method	Clean	Curr.	Stale	p50 (ms)	p95 (ms)
PFM (fresh)	.875	1.000	.000	0.06	0.09
PFM (snapshot)	.875	1.000	.000	≈ 0	≈ 0
BM25 + validity	.758	.892	.000	0.02	0.03
BM25 + recency	.212	.863	.633	0.04	0.08
BM25	.188	.858	.692	0.03	0.06
Dense (MiniLM)	.196	.975	.779	15.4	37.1
Hybrid (RRF)	.192	.879	.771	14.8	30.1

Table 2: Retrieval baselines on the update benchmark under identical corpus, device, and budgets. Clean intervals: PFM (.827, .911); BM25+validity (.700, .808); all validity-agnostic arms are below .27. Dense corpus encoding at build time takes 13.7 s; the reported latency is query-side only.

near-duplicate admission interacts destructively with verbatim revisions (only 87.1 percent of current values are indexed in the keyless arm, because a revised sentence is absorbed as a re-observation of the old fact); cancellations are the keyless arm’s worst case (it surfaces the cancelled time on 85.7 percent of those queries, where the keyed arm returns the cancellation instead); and a tight budget suppresses keyless stale exposure only by truncating current facts as well. Budget pressure is not a staleness defense. The keyed arm’s residual errors are wrong-person exposures (.125 at $B=800$, .000 at $B=240$), concentrated on the engineered ambiguous queries, where the correct fact ranks first but a same-first-name competitor also enters one of the six slots. Identity exposure under ambiguity, not staleness, is the remaining failure mode; the completion-level study below measures its downstream effect.

Retrieval Baselines under the Serving Deadline

We compare PFM against standard retrieval families on the same corpus, device, and budgets (the update benchmark’s filler corpus: 1,160 active facts, plus retained superseded facts for validity-agnostic arms; $k=6$, $B=800$). Arms: plain BM25 over all facts; BM25 with PFM’s recency decay; BM25 restricted to currently valid facts (the same validity information PFM uses, without field weighting); a local all-MiniLM-L6-v2 dense retriever (corpus embeddings precomputed, only query encoding on the serving path); a sparse–dense RRF hybrid; and PFM in fresh and snapshot modes. Latency is the median of three timed runs per query after warmup, around the full retrieval call on one thread.

Table 2 and Figure 2 support three conclusions. *Temporal correctness is set by the write path, not the ranker*: both validity-aware arms exceed .75 clean while every validity-agnostic arm stays below .27, and giving plain BM25 the same validity bits recovers most of the gap; PFM’s field weighting adds the rest, lifting current-value recall from .892 to 1.000 on the associative and participant-dependent queries. *Representational power does not substitute for temporal state*: the dense retriever has the best current-value recall (.975) yet the worst stale exposure (.779), because embedding similarity ranks every version of a revised fact near the query. *The deadline separates methods that accu-*

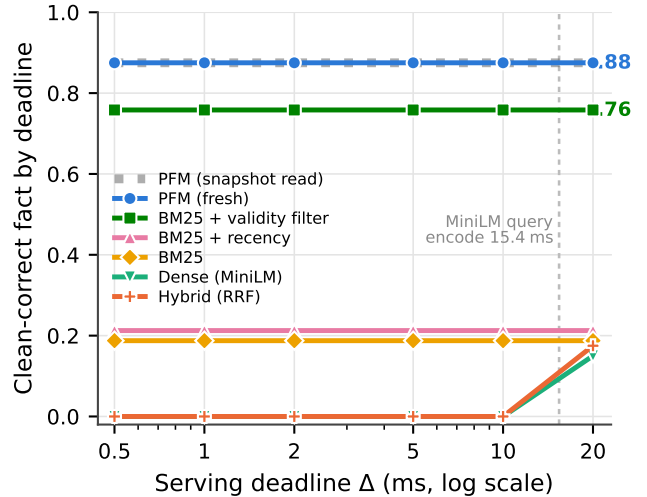


Figure 2: Clean correct retrieval by serving deadline on the update benchmark. PFM reaches its full clean rate by 0.5 ms, while dense and hybrid retrieval cannot meet deadlines below their query-encoding cost.

racy alone cannot: MiniLM query encoding costs 15.4 ms, so dense and hybrid deliver zero clean facts within a 10 ms deadline (under 18 percent within 20 ms), whereas PFM’s fresh path serves its full clean rate from 0.5 ms and the snapshot path is deadline-independent by construction. Deadline-constrained clean retrieval thus requires both validity-aware construction and a serving path whose cost is below the deadline.

Completion-Level Evidence with a Frozen Local Model

Fact availability establishes that information reached the prompt, not that it changed the completion. We therefore replay a stratified sample of 120 benchmark queries (weighted toward revised and ambiguous chains) through a frozen local model, qwen2:7b served by ollama on the same machine, with temperature 0 and identical decoding across arms. Each request assembles the same prompt the assistant would use, including typing context, participant, and the arm’s retrieved memory block, and the generated completion is judged by normalized containment: *correct* when it contains the current value, *stale* when it instead contains a superseded value, *wrong-person* when it contains a same-first-name competitor’s value, and *other* otherwise. We also record time to first streamed token (TTFT).

Table 3 shows that prompt-side differences propagate to generations. Without memory, the model produces the labeled value on none of 120 requests; every completion is a guess. With PFM’s block, 92.5 percent of completions contain the current value and none contain a stale or wrong-person value. With plain BM25’s block, 51.7 percent are correct and 35.8 percent reproduce a superseded value: stale facts in the prompt become stale text in the output at a rate close to the block’s stale-exposure rate. Notably, wrong-

Memory	Correct	Stale	Wrong pers.	Other	TTFT p50 (ms)
None	.000	.000	.000	1.000	296
PFM	.925	.000	.000	.075	1194
BM25 (plain)	.517	.358	.008	.117	1036

Table 3: Frozen-LLM replay (qwen2:7b, 120 queries, temperature 0). PFM correct 95% CI: (.864, .960); plain-BM25 stale CI: (.278, .447).

person exposure does not translate into wrong-person generation: although a same-first-name competitor’s fact enters the prompt on 12.5 percent of benchmark queries, the model selects the correctly attributed fact in every sampled case, so prompt-level exposure and completion-level error are related but not identical. The injected block also carries a prefill cost: time to first token rises from 296 ms to about 1.2 s with an 800-character block on this 7B CPU configuration; per-outcome intervals and the full judging breakdown are in the supplement. These results are specific to one model, one decoding configuration, and a containment judge; they establish that retrieved facts change completions, not that users accept them.

Latency and scaling. On the deployed replay store, measured on a single Apple Silicon thread, median retrieval latency is 0.011 ms at 270 facts (p95 0.024 ms) and 1.81 ms at 20,000 facts (p95 2.03 ms). On a deliberately dense synthetic distribution, fresh retrieval reaches 12.5 ms at 20,000 facts and 81 ms at 100,000 facts. Snapshot serving keeps the completion path to a bounded read regardless of store size, at the cost of freshness. The supplement reports the full scaling grid, hardware and timing boundaries, update cost, component ablations, and detailed update-benchmark breakdowns.

Limitations

The controlled benchmark and frozen-model replay use template-generated corpora with globally unique value tokens. Real personal text is noisier, paraphrases extend beyond the benchmark’s fixed template family, and completion-level results come from one model, one decoding configuration, and a containment-based judge. The experiments therefore establish retrieval and generation behavior under controlled conditions, not live user benefit.

Temporal correctness also depends on assigning revisions to the same mutable slot. Proposition 1 applies after correct key assignment, and the deployed assistant has not yet adopted the key-emitting rules evaluated in Table 1. Identity ambiguity remains at retrieval time: a same-first-name competitor enters 12.5% of $B=800$ benchmark prompts, although no wrong-person output appears in frozen-model replay. Graph and recency ablations do not change availability on the 25 deployed replay scenarios. Finally, all latency measurements use one Apple Silicon machine, and fresh retrieval reaches 81 ms at 100,000 facts on the dense synthetic distribution; snapshot serving avoids that completion-path delay by accepting older retrieval state.

Conclusion

Retrieval relevance alone does not determine whether personal memory is useful: a retrieved fact can be about the wrong person, obsolete, or too late to affect generation. PFM makes these failure modes explicit in both memory construction and serving. On the controlled update benchmark, keyed supersession reduces observed stale exposure from 66.2% to zero at $B=800$; matched baselines show that validity filtering accounts for most of the clean-retrieval gain, while participant-aware PFM raises clean retrieval to .875. The serving comparison shows a separate latency constraint: dense retrieval attains strong current-value recall but misses interactive deadlines because of query encoding, whereas PFM reaches its full clean rate by 0.5 ms. In frozen-7B replay, these prompt-side differences carry into generated text, with 92.5% correct completions for PFM and 35.8% stale completions for plain BM25. Evaluating identity, validity, and serving time separately exposes failures that aggregate retrieval accuracy can conceal.

References

- Banerjee, P.; Moshtaghi, M.; Subramanian, S.; Misra, A.; and Chadha, A. 2026. APEX-MEM: Agentic Semi-Structured Memory with Temporal Reasoning for Long-Term Conversational AI. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16470–16489.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Gutiérrez, B. J.; Shu, Y.; Gu, Y.; Yasunaga, M.; and Su, Y. 2024. HippoRag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37: 59532–59569.
- Huang, Z.; Tian, Z.; Guo, Q.; Zhang, F.; Zhou, Y.; Jiang, D.; Xie, Z.; and Zhou, X. 2025. Licomemory: Lightweight and cognitive agentic memory for efficient long-term reasoning. *arXiv preprint arXiv:2511.01448*.
- Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13851–13870.
- Omri, Y.; Gan, Z.; Broveak, Z.; Geens, R.; He, Z.; Pentland, A.; Verhelst, M.; Weissman, T.; and Tambe, T. 2026. Agent Memory: Characterization and System Implications of Stateful Long-Horizon Workloads. *arXiv preprint arXiv:2606.06448*.
- Packer, C.; Fang, V.; Patil, S.; Lin, K.; Wooders, S.; and Gonzalez, J. 2023. MemGPT: towards LLMs as operating systems.
- Rasmussen, P.; Paliychuk, P.; Beauvais, T.; Ryan, J.; and Chalef, D. 2025. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Robertson, S.; and Zaragoza, H. 2009. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc.

Robertson, S.; Zaragoza, H.; and Taylor, M. 2004. Simple BM25 extension to multiple weighted fields. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, 42–49.

Su, M.; Guo, Y.; Hou, Z.; Bai, L.; Li, Z.; Zhang, Y.; Yin, G.; Lin, W.; Jin, X.; Guo, J.; et al. 2026. Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents. *arXiv preprint arXiv:2601.07468*.

Wang, Z.; Li, Z.; Jiang, Z.; Tu, D.; and Shi, W. 2024. Crafting personalized agents through retrieval-augmented generation on editable memory graphs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 4891–4906.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.

Xia, M.; Zhang, X.; Dixit, S.; Harimurugan, P.; Wang, R.; Rühle, V.; Sim, R.; Bansal, C.; and Rajmohan, S. 2026. Memora: A harmonic memory representation balancing abstraction and specificity. In *Forty-third International Conference on Machine Learning*.

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38: 17577–17604.

Yuan, A.; Zhang, H.; and Nazarian, S. 2026. AgentIR: A Workload-Adaptive Cascade Retrieval Substrate for Long-Term Conversational Memory. *arXiv preprint arXiv:2605.25092*.

Zhang, J.; Zhang, C.; Chen, S.; Huang, Z.; Zheng, P.; Wang, Z.; Guo, P.; Mo, F.; Bae, S.-H.; Zou, J.; et al. 2026. Lightweight llm agent memory with small language models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 12914–12929.

Zhao, X.; Ma, Q.; Zhang, Y.; Lou, H.; Cheng, G.; Deng, S.; and Yin, J. 2025. AME: An Efficient Heterogeneous Agentic Memory Engine for Smartphones. *arXiv preprint arXiv:2511.19192*.

Zhou, W.; Zhou, X.; Han, S.; Xu, H.; Li, G.; Li, Z.; Xiong, F.; and Wu, F. 2026. Are We Ready For An Agent-Native Memory System? *arXiv preprint arXiv:2606.24775*.